

Documentation Terraform

Terraform– Projet d'automatisation

1. Présentation du projet

Ce projet consiste à automatiser le déploiement de **machines virtuelles** sur un environnement **Proxmox** à l'aide de l'outil d'infrastructure as code **Terraform**.

L'objectif principal est de simplifier, standardiser et accélérer la création de machines virtuelles pour répondre aux besoins d'un environnement de test. Grâce à Terraform, le processus de déploiement devient répétable, contrôlable et versionnable, ce qui améliore la gestion de l'infrastructure et réduit le risque d'erreurs humaines.

Le but de ce projet est de fournir une interface mobile légère, moderne et facile à utiliser pour surveiller les performances et la santé de machines virtuelles.

2. Dans ce projet :

- Terraform est utilisé pour **décrire** et **provisionner** l'infrastructure virtuelle (machines, ressources réseaux, configurations) sur un serveur **Proxmox VE**.
- Les **machines virtuelles** sont automatiquement créées avec les paramètres souhaités (nombre de CPU, quantité de RAM, espace disque, etc.).
- Le tout est orchestré par des fichiers de configuration (.tf) codés en **HCL** (HashiCorp Configuration Language).
- Le but est de permettre un déploiement rapide, fiable et automatisé, sans passer par l'interface manuelle de Proxmox.

3. Qu'est-ce que Terraform ?

Terraform est un outil **open-source** développé par **HashiCorp** qui permet de **décrire** une infrastructure sous forme de **code** (Infrastructure as Code – IaC).

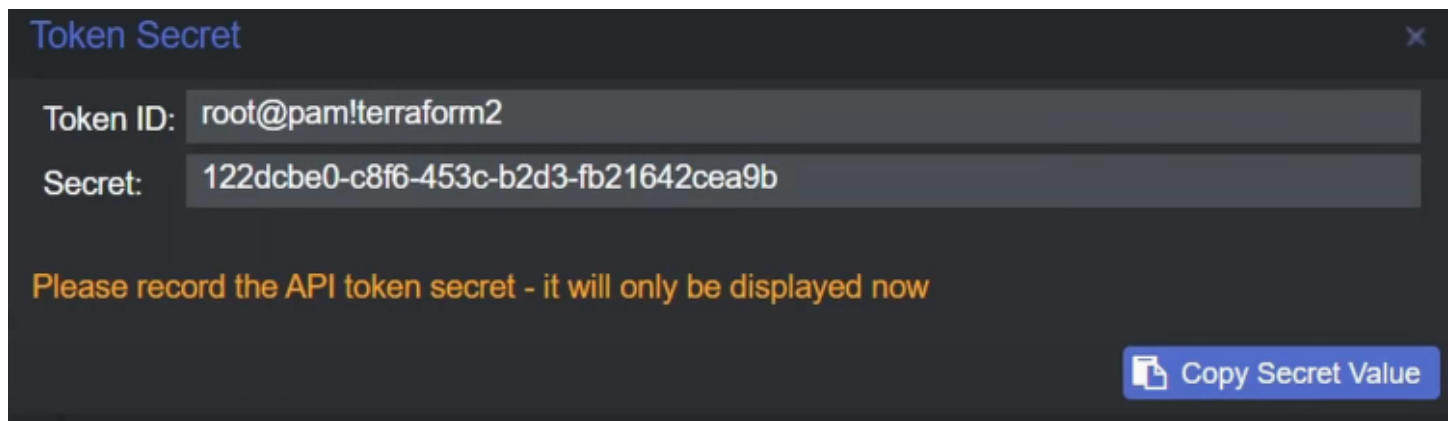
Avec Terraform, il est possible de :

- Définir des serveurs, bases de données, réseaux, services cloud... sous forme de fichiers texte.
- Automatiser le **déploiement**, la **modification**, et la **destruction** de l'infrastructure.
- Avoir une **traçabilité** et un **versionning** de toute l'infrastructure.
- Travailler de manière **collaborative** sur les infrastructures grâce au contrôle de version (ex: Git).

Dans ce projet, Terraform est utilisé avec le **provider Proxmox**, qui permet à Terraform de piloter directement les API de Proxmox pour créer et gérer les machines virtuelles.

4 .Configuration sur Proxmox:

Pour que Terraform puisse communiquer avec Proxmox, on utilise l'API REST de Proxmox. Il faut donc créer un utilisateur spécifique.



5. Fichiers Terraform:

provider.tf:

```

1  terraform {
2
3      required_version = ">= 0.13.0"
4
5      required_providers {
6          proxmox = {
7              source = "telmate/proxmox"
8              version = ">= 2.9.14"
9          }
10     }
11 }
12
13 variable "proxmox_api_url" {
14     type = string
15 }
16
17 variable "proxmox_api_token_id" {
18     type = string
19 }
20
21 variable "proxmox_api_token_secret" {
22     type = string
23 }
24
25 provider "proxmox" {
26
27     pm_api_url = var.proxmox_api_url
28     pm_api_token_id = var.proxmox_api_token_id
29     pm_api_token_secret = var.proxmox_api_token_secret
30 }

```

credential.auto.tfvars :

```

1  proxmox_api_url = "https://10.0.0.220:8006/api2/json"
2  proxmox_api_token_id = "root@pam!terraform1"
3  proxmox_api_token_secret = "942860a8-cb09-4587-a822-93e16ebe55b9"

```

vm.tf :

```

1  resource "proxmox_vm_qemu" "srv_test" {
2
3
4
5      count = 2
6      desc = "Windows server"
7      vmid = "102"
8      target_node = "proxmox"
9
10     agent = 1
11
12     clone = "template-test-1"
13     cores = 1
14     sockets = 2
15     cpu = "host"
16     memory = "4096"
17     name = var.vm_names[count.index]
18
19
20     disk {
21         storage = "local-lvm"
22         type = "virtio"
23         size = "60G"
24     }
25
26
27     # Configuration réseau
28     network {
29         model = "virtio"
30         bridge = "vmbbr0"
31
32     }
33
34     ipconfig0 = "${var.vm_gateways[count.index]}"
35
36
37 }

```

6.Initialiser et déployer:

Dans le terminal à l'emplacement du dossier de configuration de Terraform on tape:

terraform init

(pour initialiser le projet et télécharger les providers)

Puis :

“terraform plan”

(pour voir ce qui va être créé)

Et enfin :

terraform apply

(pour appliquer et créer la VM sur Proxmox)

Exemple d'initialisation et de déploiement sur Proxmox:

```
(base) → test git:(master) x terraform init
```

Initializing the backend...

Initializing provider plugins...

- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v5.45.0...
- Installed hashicorp/aws v5.45.0 (signed by HashiCorp)

Terraform has created a lock file `.terraform.lock.hcl` to record the provider selections it made above. Include this file in your version control repository so that Terraform can guarantee to make the same selections by default when you run `"terraform init"` in the future.

Terraform has been successfully initialized!

```
(base) → test git:(master) x terraform plan
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

```
# aws_instance.ec2_instance will be created
+ resource "aws_instance" "ec2_instance" {
  + ami                  = "ami-051f8a213df8bc089"
  + get_password_data    = false
  + instance_type        = "t2.micro"
  + source_dest_check    = true
  + user_data_replace_on_change = false
  ...
}
```

Plan: 1 to add, 0 to change, 0 to destroy.